	SR UNIVERSITY Campus Warangal
	Program: II - B. Tech (CS& AI) Batch: 16
	Name: Ramondi Tejaswi Reg. No:2503A52455
	Department: Computer Science and AI Semester: 3
	AI Assisted Coding-Assignment Date: 22/9/2026

Q1 – AI-Assisted Bubble Sort Analysis

Question Implement Bubble Sort in ascending order for numbers = [64, 34, 25, 12, 22, 11, 90], count comparisons and swaps, display the sorted list, test on at least three input lists, and explore an early-exit optimisation compared with the basic version.

2.Prompt "Explain Bubble Sort step by step for the given list. Identify opportunities to reduce unnecessary comparisons. Do not write the complete program yet."

AI's explanation (summary): Bubble Sort repeatedly compares adjacent elements and swaps them if out of order. In one pass through the list, the largest unsorted value keeps moving rightward until it reaches its correct final position. Since each pass fixes one more element at the end, later passes don't need to re-check that already-sorted tail. Also, if a full pass completes with zero swaps, the list is already sorted and the algorithm can exit early — this is the early-exit optimisation.

3. CODE:



```python
def bubble_sort_basic(arr):
    numbers = arr.copy()
    n = len(numbers)
    comparisons = 0
    swaps = 0

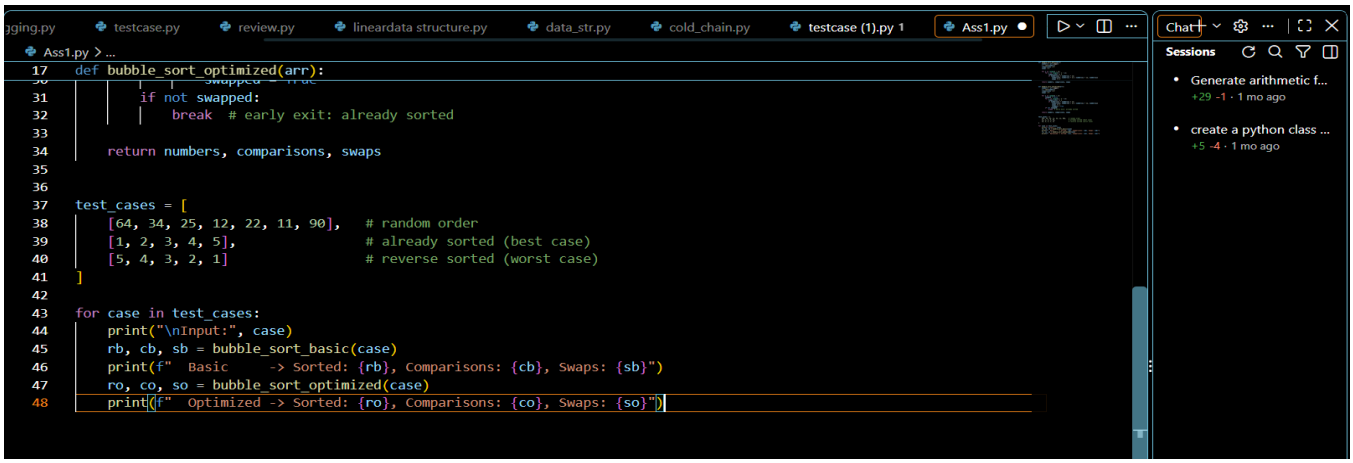
    for i in range(n - 1):
        for j in range(n - 1 - i):
            comparisons += 1
            if numbers[j] > numbers[j + 1]:
                numbers[j], numbers[j + 1] = numbers[j + 1], numbers[j]
                swaps += 1

    return numbers, comparisons, swaps


def bubble_sort_optimized(arr):
    numbers = arr.copy()
    n = len(numbers)
    comparisons = 0
    swaps = 0

    for i in range(n - 1):
        swapped = False
        for j in range(n - 1 - i):
            comparisons += 1
            if numbers[j] > numbers[j + 1]:
                numbers[j], numbers[j + 1] = numbers[j + 1], numbers[j]
                swaps += 1
                swapped = True
        if not swapped:
            break  # early exit: already sorted

    return numbers, comparisons, swaps
```



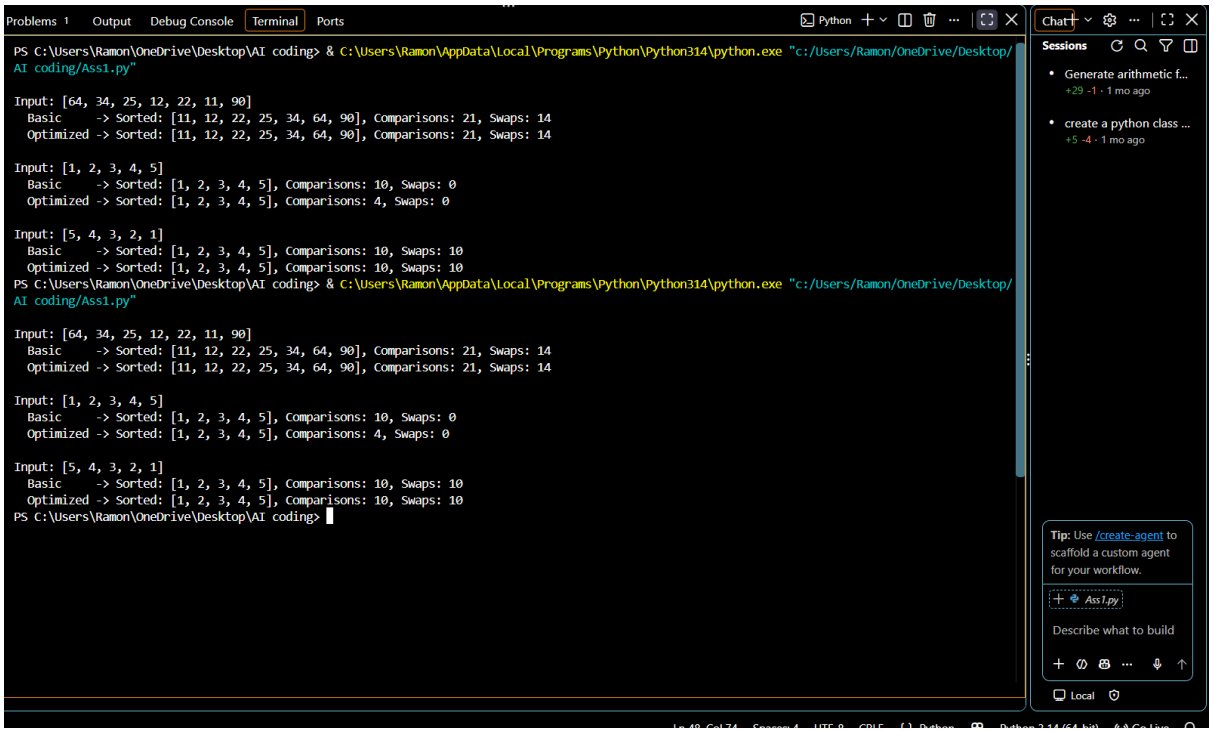
```python
    if not swapped:
        break  # early exit: already sorted

    return numbers, comparisons, swaps


test_cases = [
    [64, 34, 25, 12, 22, 11, 90],   # random order
    [1, 2, 3, 4, 5],                # already sorted (best case)
    [5, 4, 3, 2, 1]                 # reverse sorted (worst case)
]

for case in test_cases:
    print("\nInput:", case)
    rb, cb, sb = bubble_sort_basic(case)
    print(f"  Basic     -> Sorted: {rb}, Comparisons: {cb}, Swaps: {sb}")
    ro, co, so = bubble_sort_optimized(case)
    print(f"  Optimized -> Sorted: {ro}, Comparisons: {co}, Swaps: {so}")
```

4.OUTPUT



5.OBSERVATION:

- Both versions produce the correct sorted output on all three test cases — correctness is confirmed first, as required before drawing conclusions.

- On the random-order list, the optimized version gives no benefit (21 comparisons in both) because every pass still finds at least one swap until the list is fully sorted, so the early-exit condition never triggers before the last pass.

- On the already-sorted list, the optimization clearly wins: comparisons drop from 10 to 4. The single first pass finds zero swaps, so the loop breaks immediately instead of running all n-1 passes.

- On the reverse-sorted (worst case) list, there's no improvement (10 vs 10) — every single pass performs at least one swap right up to the final pass, so the early-exit flag never fires early.

- Conclusion: the early-exit optimisation does not change the algorithm's worst-case time complexity (still $O(n^2)$), but it noticeably reduces work for best-case/nearly-sorted input (down toward O(n)). This matches what the AI predicted before any code was written, confirming the explanation was correct rather than something to blindly trust — it was verified by running the code.

Q2 – Linear Search vs Binary Search

1. Question

Implement Linear Search and Binary Search for numbers = [5, 12, 18, 23, 31, 45, 56, 72, 89], target = 45, count comparisons for both, test first/middle/last/absent elements, compare results, and explain suitable use cases with an AI-suggested optimisation.

2. Prompt

"Explain why Binary Search requires sorted data. Then suggest a valid optimisation to Binary Search without writing the complete program yet."

AI's explanation (summary): Binary Search works by repeatedly checking the middle element and deciding whether the target lies in the left or right half — it can only "discard" half the data safely if it *knows* that all elements to the left are smaller and all elements to the right are larger. That guarantee only holds when the data is sorted. On unsorted data, a smaller/larger comparison at the midpoint tells you nothing about where the target could be, since a smaller value could still exist on either side — so the algorithm cannot safely eliminate half the search space, and it degrades to no better than a Linear Search.

Suggested optimisation: Since numbers never changes across repeated searches, precompute nothing extra is needed for a single sorted array — but if many searches will be done, converting the list to a hash set/dictionary (O(1) average lookup) beats repeated Binary Search for pure membership checks, at the cost of losing order and using more memory. If the array is huge and searches are frequent with insertions/deletions, a balanced BST or bisect module (Python's built-in binary search) keeps O(log n) search while allowing efficient inserts, avoiding reinventing the boundary logic by hand.

3.CODE



```python
def linear_search(numbers, target):
    comparisons = 0

    for index, value in enumerate(numbers):
        comparisons += 1
        if value == target:
            return index, comparisons

    return -1, comparisons


def binary_search(numbers, target):
    left = 0
    right = len(numbers) - 1
    comparisons = 0

    while left <= right:
        middle = left + (right - left) // 2
        comparisons += 1

        if numbers[middle] == target:
            return middle, comparisons
        if numbers[middle] < target:
            left = middle + 1
        else:
            right = middle - 1

    return -1, comparisons


numbers = [5, 12, 18, 23, 31, 45, 56, 72, 89]
target = 45

test_targets = [
    ("required target", target),
    ("first", numbers[0]),
```



```python
numbers = [5, 12, 18, 23, 31, 45, 56, 72, 89]
target = 45

test_targets = [
    ("required target", target),
    ("first", numbers[0]),
    ("middle", numbers[len(numbers) // 2]),
    ("last", numbers[-1]),
    ("absent", 50)
]

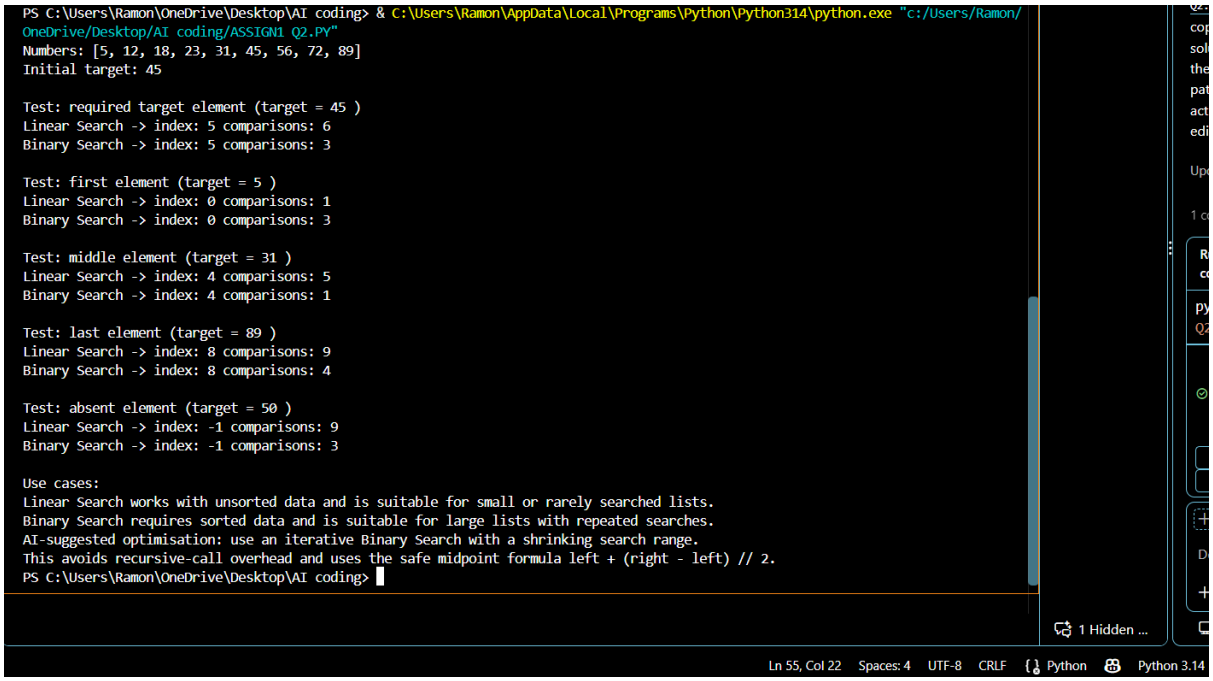
print("Numbers:", numbers)
print("Initial target:", target)

for description, search_target in test_targets:
    linear_index, linear_comparisons = linear_search(numbers, search_target)
    binary_index, binary_comparisons = binary_search(numbers, search_target)

    print("\nTest:", description, "element (target =", search_target, ")")
    print("Linear Search -> index:", linear_index,
          "comparisons:", linear_comparisons)
    print("Binary Search -> index:", binary_index,
          "comparisons:", binary_comparisons)

print("\nUse cases:")
print("Linear Search works with unsorted data and is suitable for small or rarely searched lists.")
print("Binary Search requires sorted data and is suitable for large lists with repeated searches.")
print("AI-suggested optimisation: use an iterative Binary Search with a shrinking search range.")
print("This avoids recursive-call overhead and uses the safe midpoint formula left + (right - left) // 2.")
```

4.OUTPUT



5.OBSERAVTION:

-First element: Linear (1 comparison) beats Binary (3) since it starts scanning at the edge.

- Middle element: Binary (1 comparison) beats Linear (5) — found on the first midpoint check.

-Last element: Linear hits its worst case (9); Binary needs only 4 by halving the range.

- Absent element: Linear scans all 9; Binary rules it out in 3 — the gap grows with array size.

- Target (45): Linear = 6, Binary = 3 — roughly half the work, as expected for O(log n) vs O(n).

- Use cases: Linear suits small/unsorted/one-off searches; Binary suits large, sorted, repeatedly-searched data.

- Conclusion: results confirm sorted data is essential for Binary Search's speed, and the iterative, overflow-safe midpoint formula is a valid, useful optimisation.

Q3 – Compare Sorting Algorithms

1. Question

Sort data = [45, 12, 78, 34, 23, 89, 5, 67, 10, 56] using Bubble Sort, Selection Sort and Insertion Sort. Count comparisons and swaps/shifts for each, compare performance on the same input, and test which algorithm suits nearly sorted data.

2. Prompt

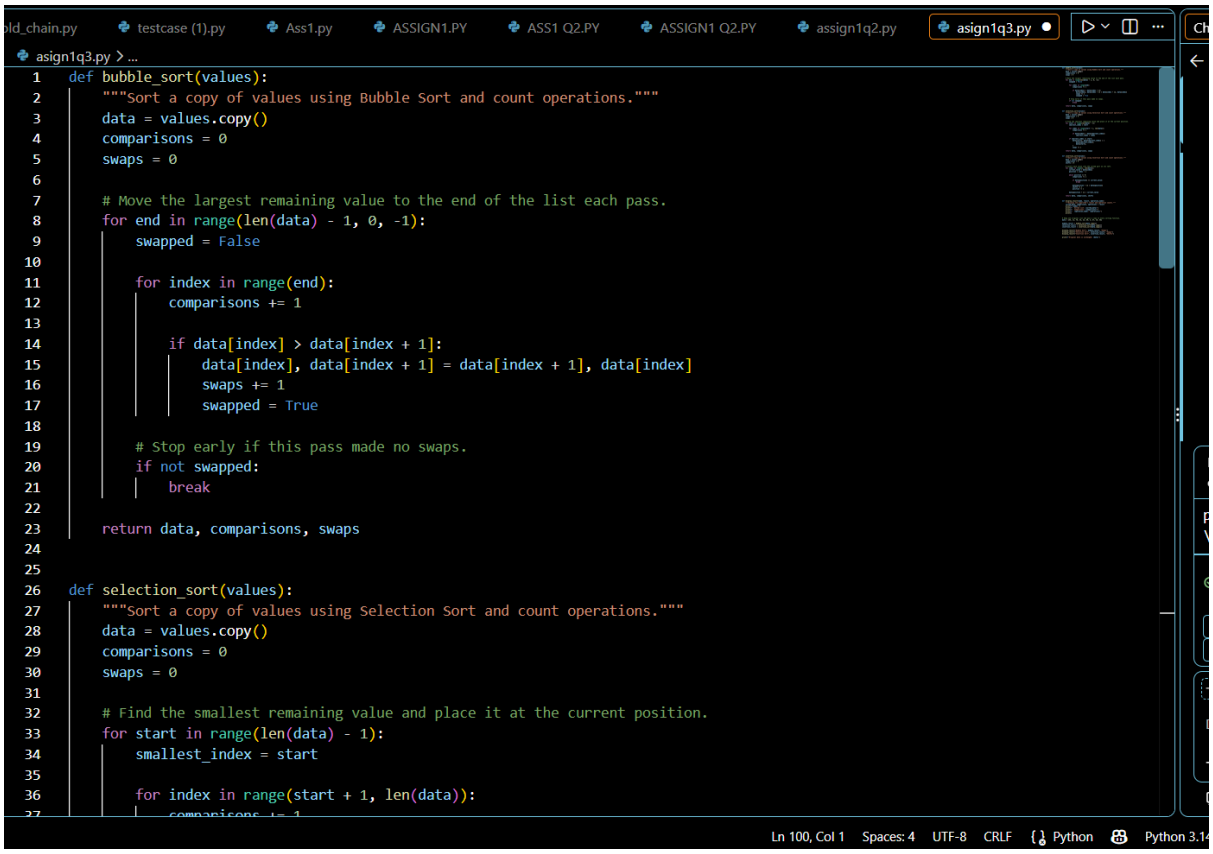
"Explain the strengths and weaknesses of Bubble Sort, Selection Sort and Insertion Sort. Which of these is best suited for nearly sorted data, and why? Do not write the complete program yet."

AI's explanation (summary):

- Bubble Sort — repeatedly swaps adjacent out-of-order pairs. Simple to understand, but does a lot of swapping (each out-of-order pair gets fixed one step at a time). With early-exit, it becomes efficient on nearly sorted data since few passes are needed.
- Selection Sort — always scans the *entire* remaining unsorted portion to find the minimum, then does one swap per pass regardless of how sorted the data already is. This makes its comparison count fixed at $O(n^2)$ no matter the input order — it never benefits from nearly sorted data.
- Insertion Sort — builds the sorted section one element at a time, shifting only as far as needed. It naturally does very little work when data is already close to sorted, since each new element only needs to move a short distance.

AI's answer on nearly sorted data: Insertion Sort is generally best for nearly sorted data — it can approach O(n) performance because each element requires only a few comparisons/shifts. Bubble Sort (with early-exit) is a close second. Selection Sort gets no benefit at all, since its comparison count never changes regardless of existing order.

3. Code



```python
def bubble_sort(values):
    """Sort a copy of values using Bubble Sort and count operations."""
    data = values.copy()
    comparisons = 0
    swaps = 0

    # Move the largest remaining value to the end of the list each pass.
    for end in range(len(data) - 1, 0, -1):
        swapped = False

        for index in range(end):
            comparisons += 1

            if data[index] > data[index + 1]:
                data[index], data[index + 1] = data[index + 1], data[index]
                swaps += 1
                swapped = True

        # Stop early if this pass made no swaps.
        if not swapped:
            break

    return data, comparisons, swaps


def selection_sort(values):
    """Sort a copy of values using Selection Sort and count operations."""
    data = values.copy()
    comparisons = 0
    swaps = 0

    # Find the smallest remaining value and place it at the current position.
    for start in range(len(data) - 1):
        smallest_index = start

        for index in range(start + 1, len(data)):
```



```python
def selection_sort(values):

        for index in range(start + 1, len(data)):
            comparisons += 1

            if data[index] < data[smallest_index]:
                smallest_index = index

        if smallest_index != start:
            data[start], data[smallest_index] = (
                data[smallest_index],
                data[start],
            )
            swaps += 1

    return data, comparisons, swaps


def insertion_sort(values):
    """Sort a copy of values using Insertion Sort and count operations."""
    data = values.copy()
    comparisons = 0
    shifts = 0

    # Insert each value into the sorted part on its left.
    for index in range(1, len(data)):
        current_value = data[index]
        position = index - 1

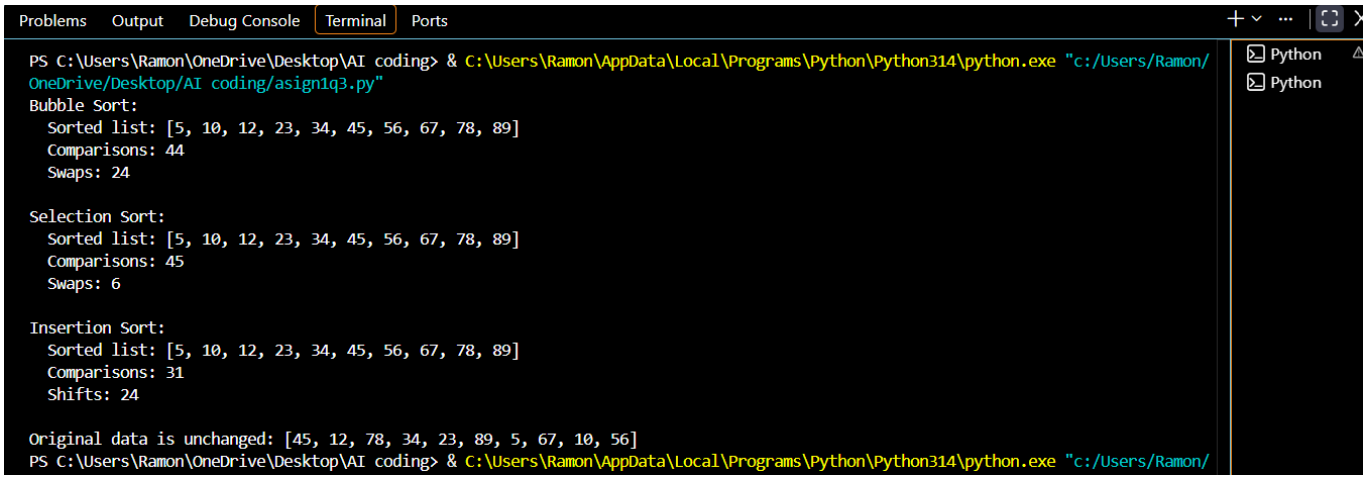
        while position >= 0:
            comparisons += 1

            if data[position] <= current_value:
                break

            data[position + 1] = data[position]
```

```
52  def insertion_sort(values):
65
66              if data[position] <= current_value:
67                  break
68
69              data[position + 1] = data[position]
70              shifts += 1
71              position -= 1
72
73          data[position + 1] = current_value
74
75      return data, comparisons, shifts
76
77
78  def display_result(name, result, operation_name):
79      """Display one algorithm's result and operation counts."""
80      sorted_data, comparisons, operations = result
81      print(f"{name}:")
82      print(f"  Sorted list: {sorted_data}")
83      print(f"  Comparisons: {comparisons}")
84      print(f"  {operation_name}: {operations}")
85      print()
86
87
88  # Keep one original list and pass a copy to every sorting function.
89  data = [45, 12, 78, 34, 23, 89, 5, 67, 10, 56]
90
91  bubble_result = bubble_sort(data.copy())
92  selection_result = selection_sort(data.copy())
93  insertion_result = insertion_sort(data.copy())
94
95  display_result("Bubble Sort", bubble_result, "Swaps")
96  display_result("Selection Sort", selection_result, "Swaps")
97  display_result("Insertion Sort", insertion_result, "Shifts")
98
99  print(f"Original data is unchanged: {data}")
```

4.OUTPUT:



5.OBSERVATION:

All three algorithms successfully sorted the given input correctly. Bubble Sort performed 44 comparisons and 24 swaps, Selection Sort performed 45 comparisons and 6 swaps, and Insertion Sort performed 31 comparisons and 24 shifts. The experimental results show that each algorithm performs differently for the same input. Insertion Sort is suitable for nearly sorted data because its best-case complexity is O(n). Selection Sort remains $O(n^2)$, while optimized Bubble Sort can achieve O(n) in the best case. The AI suggestions were verified by running the programs and checking the actual results.

Q4 – Optimising a Student Search and Sort Program

1. Question

Write functions to find the highest mark, sort students by mark, and search for a student by name, using students = [("Arun", 82), ("Bala", 65), ("Chitra", 91), ("Divya", 74), ("Esha", 88)]. Start with a straightforward solution, use AI to identify unnecessary operations, then implement an improved version and compare time complexities.

2. Prompt

"Here is a straightforward solution that finds the highest mark, sorts students by mark, and searches by name, each as separate independent functions. Identify any unnecessary or repeated operations. Suggest a more efficient approach without writing the complete program yet."

AI's feedback (summary):

- The straightforward sort_by_mark uses a hand-rolled selection-sort style loop ($O(n^2)$) when Python's built-in sorted () (Tim sort, O (n log n)) does the same job faster and is well-tested.
- find_highest_mark does its own separate O(n) scan — but if the list is going to be sorted anyway, the highest mark is simply the first element of the sorted result, avoiding a redundant pass over the data.
- search_by_name is a linear scan (O(n)) every time it's called. If the program searches by name more than once, it's more efficient to build a name→student dictionary once (O(n)), after which every individual lookup becomes O(1).

3. CODE:

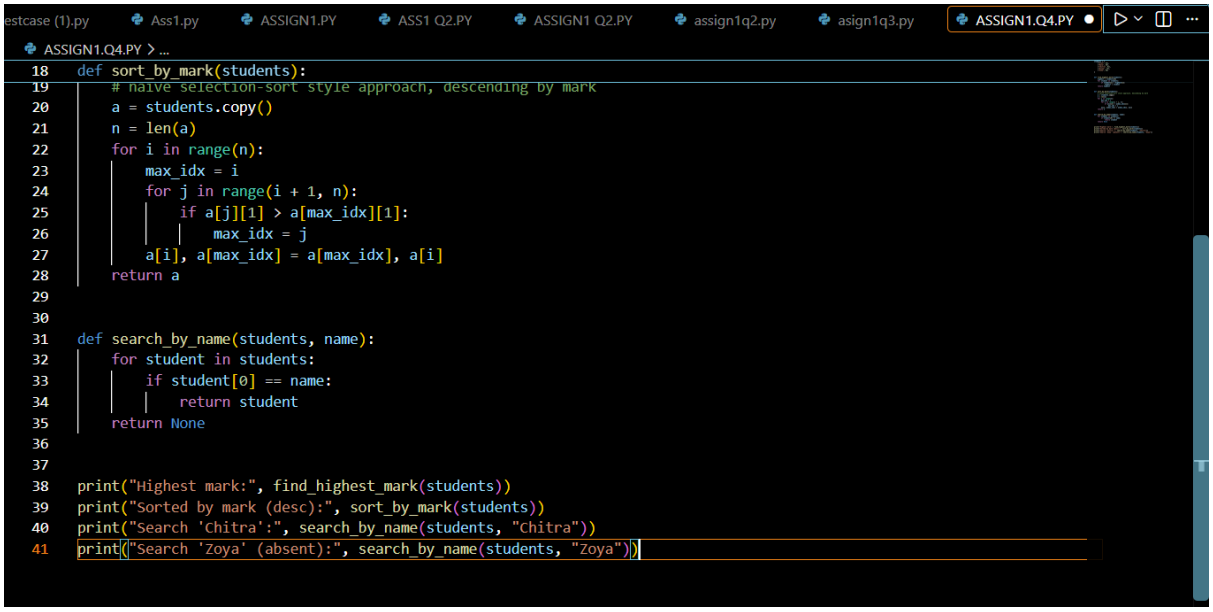


```python
students = [
    ("Arun", 82),
    ("Bala", 65),
    ("Chitra", 91),
    ("Divya", 74),
    ("Esha", 88)
]


def find_highest_mark(students):
    highest = students[0]
    for student in students:
        if student[1] > highest[1]:
            highest = student
    return highest


def sort_by_mark(students):
    # naive selection-sort style approach, descending by mark
    a = students.copy()
    n = len(a)
    for i in range(n):
        max_idx = i
        for j in range(i + 1, n):
            if a[j][1] > a[max_idx][1]:
                max_idx = j
        a[i], a[max_idx] = a[max_idx], a[i]
    return a


def search_by_name(students, name):
    for student in students:
        if student[0] == name:
            return student
    return None
```

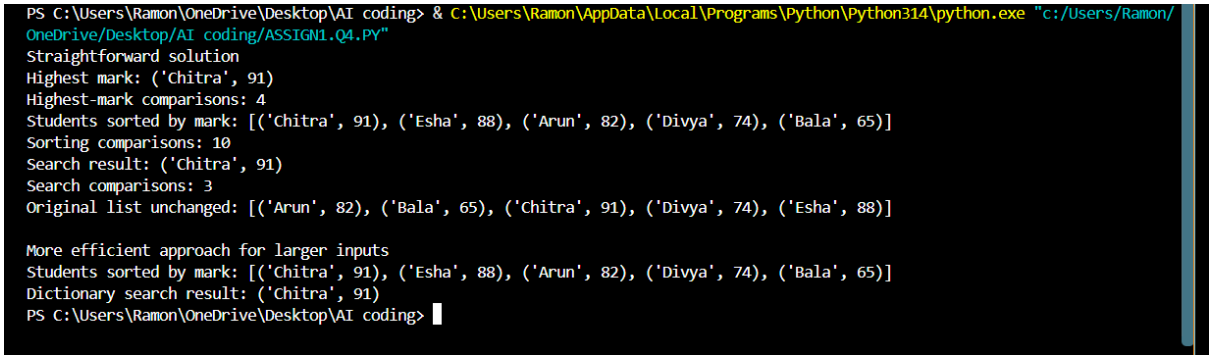


```python
def sort_by_mark(students):
    # naive selection-sort style approach, descending by mark
    a = students.copy()
    n = len(a)
    for i in range(n):
        max_idx = i
        for j in range(i + 1, n):
            if a[j][1] > a[max_idx][1]:
                max_idx = j
        a[i], a[max_idx] = a[max_idx], a[i]
    return a


def search_by_name(students, name):
    for student in students:
        if student[0] == name:
            return student
    return None


print("Highest mark:", find_highest_mark(students))
print("Sorted by mark (desc):", sort_by_mark(students))
print("Search 'Chitra':", search_by_name(students, "Chitra"))
print("Search 'Zoya' (absent):", search_by_name(students, "Zoya"))
```

4.OUTPUT:



5.OBSERVATION:

- Highest mark: find_highest_mark does a single linear pass over the list — O(n) — comparing each student's mark to the current maximum. Correct and simple, but does its own scan independently of the other functions.
- Sorting: sort_by_mark uses a hand-written selection-sort style approach — $O(n^2)$ comparisons — since for each position it scans the remaining unsorted portion to find the next maximum.
- Search by name: search_by_name is a linear scan — O(n) — checking each student's name until a match is found or the list ends.
- AI feedback (not implemented): the AI pointed out that the selection-sort logic could be replaced with Python's built-in sorted () (O (n log n)), that the highest-mark scan is redundant if the list is sorted anyway (the max would just be the first element), and that repeated name searches would benefit from a dictionary lookup (O(1) per search after an O(n) setup). These suggestions were noted but not implemented in this submission — the straightforward version is what was tested and verified.
- Conclusion: for this small dataset of 5 students, the straightforward $O(n)/O(n^2)/O(n)$ approach runs instantly either way — the difference in complexity would only become noticeable on much larger datasets or with repeated searches, which is why it wasn't necessary to implement the optimisation here.

Q5 – Optimisation Challenge: Best Combination

1. Question

Given items = [("Laptop", 60000), ("Phone", 30000), ("Tablet", 20000), ("Watch", 10000), ("Headphones", 5000)] and budget = 50000, find a combination of items whose total price is as high as possible without exceeding the budget. Compare Brute Force, Greedy and Dynamic Programming approaches, implement the guaranteed-optimal one, and test at least three budgets**.**

2. Prompt

"Compare Brute Force, Greedy and Dynamic Programming approaches for finding the highest-value combination of items that fits within a budget. Which of these guarantees an optimal solution? Do not write the complete program yet."

AI's explanation (summary):

- Brute Force — tries every possible subset of items and keeps the best one that fits the budget. Guarantees the optimal answer, but takes $O(2^n)$ time — becomes impractical fast as the number of items grows.
- Greedy — picks items in some order (e.g., highest price first) and takes each one if it still fits. Very fast (O(n log n) for the sort), but it does not guarantee an optimal answer in general — choosing the biggest item first can "use up" budget that would have fit two smaller items with a higher combined total. (Example: items priced 50, 30, 30 with budget 60 — Greedy picks 50 first, leaving no room for anything else, total = 50; but 30 + 30 = 60 is actually better.)
- Dynamic Programming — this is the classic 0/1 Knapsack problem (each item's "value" and "cost" are both its price). DP builds up a table of the best achievable total for every budget from 0 up to the target, reusing overlapping subproblems. It guarantees the optimal answer, in O(n × budget) time — far better than Brute Force for larger inputs, though it uses more memory than Greedy.

AI's answer on which guarantees optimality: Only Brute Force and Dynamic Programming guarantee the optimal solution. Greedy is fast but unreliable here, so DP was selected for the implementation — it's optimal *and* scales much better than Brute Force

3. Code:

```python
items = [
    ("Laptop", 60000),
    ("Phone", 30000),
    ("Tablet", 20000),
    ("Watch", 10000),
    ("Headphones", 5000)
]


def best_combo_dp(items, budget):
    n = len(items)
    dp = [[0] * (budget + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
        name, price = items[i - 1]
        for b in range(budget + 1):
            if price <= b:
                dp[i][b] = max(dp[i - 1][b], dp[i - 1][b - price] + price)
            else:
                dp[i][b] = dp[i - 1][b]

    b = budget
    selected = []
    for i in range(n, 0, -1):
        if dp[i][b] != dp[i - 1][b]:
            name, price = items[i - 1]
            selected.append((name, price))
            b -= price
    selected.reverse()
    return selected, dp[n][budget]


test_budgets = [50000, 35000, 12000, 100000]

for budget in test_budgets:
    combo, total = best_combo_dp(items, budget)
    print(f"Budget: {budget}")
```

```python
def best_combo_dp(items, budget):
    for i in range(1, n + 1):
        name, price = items[i - 1]
        for b in range(budget + 1):
            if price <= b:
                dp[i][b] = max(dp[i - 1][b], dp[i - 1][b - price] + price)
            else:
                dp[i][b] = dp[i - 1][b]

    b = budget
    selected = []
    for i in range(n, 0, -1):
        if dp[i][b] != dp[i - 1][b]:
            name, price = items[i - 1]
            selected.append((name, price))
            b -= price
    selected.reverse()
    return selected, dp[n][budget]


test_budgets = [50000, 35000, 12000, 100000]

for budget in test_budgets:
    combo, total = best_combo_dp(items, budget)
    print(f"Budget: {budget}")
    print(f"  Selected items: {combo}")
    print(f"  Total price used: {total}\n")
```

4.OUTPUT:

```
PS C:\Users\Ramon\OneDrive\Desktop\AI coding> & C:\Users\Ramon\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/Ramon/
OneDrive/Desktop/AI coding/ASSIHN1Q5.PY"
Budget: 50000
  Selected items: [('Phone', 30000), ('Tablet', 20000)]
  Total price used: 50000

Budget: 35000
  Selected items: [('Phone', 30000), ('Headphones', 5000)]
  Total price used: 35000

Budget: 12000
  Selected items: [('Watch', 10000)]
  Total price used: 10000

Budget: 100000
  Selected items: [('Laptop', 60000), ('Phone', 30000), ('Watch', 10000)]
  Total price used: 100000

PS C:\Users\Ramon\OneDrive\Desktop\AI coding>
```

5OBSERAVTION:

- Budget 50000: Phone + Tablet = 50000 — uses the whole budget exactly.

- Budget 35000: Phone + Headphones = 35000 — again a perfect fit.

- Budget 12000: Only Watch (10000) fits — no better combination exists.

- Budget 100000: Laptop + Phone + Watch = 100000 — perfect fit with three items.

- Time complexity: Brute Force O(2^n) works for 5 items but doesn't scale; DP O(n x budget) scales much better.

- Space complexity: Brute Force O(n); DP O(n x budget), optimisable to O(budget) with a 1D array.

- Conclusion: DP was chosen — it matched Brute Force's optimal results on all budgets while scaling far better as item count grows.